

ORALGUARD

AI-Based Oral Cancer Screening Application

UI & DATABASE DESIGN DOCUMENT

Submitted By: Yashas R. Shetty

Reg. No.: 2340263

Date: 28/02/2026

SYSTEM ARCHITECTURE

Architecture Overview

The OralGuard system follows a modular architecture consisting of a Presentation Layer, Application Layer, and Data Layer. The system is designed for efficient Content-Based Image Retrieval (CBIR) using deep learning.

- **Presentation Tier:**

- Web UI (HTML/CSS/JavaScript)
- Home Page, Risk Screener, Image Matcher, Self-Exam Guide

This tier accepts image input from user -> sends image to backend API -> displays benign and malignant match results -> shows similarity scores and ranked results

- **Application Tier:**

- FastAPI, PyTorch, Torchvision, Scikit-learn
- Image Preprocessing Module
- Feature Extraction Engine (MobileNetV2 Backbone)
- CBIR (Cosine) Similarity Engine
- Result Generator

Workflow:

User Upload → Image Preprocessing → Feature Extraction → Cosine Similarity Calculation → Separate Benign & Malignant Ranking → Return Top Matches

- **Data Tier:**

The system does not use a traditional relational database. Instead, it uses structured image folders, a precomputed feature index file, and public dataset hosting on Hugging Face.

DATA DESCRIPTION & DATASET STRUCTURE

The dataset consists of oral lesion images stored in PNG format. All images are organized under a main directory named **“oral-images.”** Within this directory, the data is divided into two sub-folders based on medical classification.

The first folder, **“benign_lesions,”** contains images of non-cancerous oral lesions. The second folder, **“malignant_lesions,”** contains images of cancerous oral lesions.

Feature Index File: Instead of storing image data in a database, the system uses a precomputed NumPy index file: *cbir_index.npy*, which contains feature vectors extracted using MobileNetV2, original dataset image paths, and image class labels (benign / malignant)

This file is loaded once at API startup and stored in memory for fast similarity search.

MODEL DESIGN

Feature Extraction Model

The system uses MobileNetV2 pretrained on ImageNet. The classifier layer is removed, and Global Average Pooling is applied. Input image size is 224 x 224 pixels. Images are resized, converted to tensors, and normalized using ImageNet mean and standard deviation.

Similarity Engine

Cosine similarity is used to compare the query image embedding with all stored embeddings. Results are separated into benign and malignant groups, ranked independently, and the top 6 matches from each category are returned.

API DESIGN

GET / → Returns API status confirmation.

POST /search → Accepts an uploaded image, extracts features, computes similarity, and returns ranked benign and malignant matches with similarity scores and public image URLs.

INTERFACE DESIGN

- Home Interface – Navigation and Overview
- Risk Screener – Questionnaire and Risk Score Display
- Image Matcher – Upload and AI Similarity Results
- Result Interface – Comparison View with Similarity Scores

PROGRAM COMPONENT STRUCTURE

- **Frontend:**

Navigation UI
Questionnaire UI
Image Upload UI

- **Backend:**

Feature Extractor
Similarity Engine
Risk Engine
Result Formatter

- **Database:**

Responses
Image Records
Match Results

DEPLOYMENT

The backend application is containerized using Docker to ensure environment consistency and dependency isolation. The Docker container is deployed and hosted on Hugging Face Spaces. The image dataset is hosted separately on Hugging Face Datasets, and public URLs are generated dynamically during result formatting.

SYSTEM FLOW

1. User uploads oral lesion image.
2. Backend preprocesses image.
3. MobileNetV2 extracts feature embedding.
4. Cosine similarity is computed.
5. Results split into benign and malignant categories.
6. Top matches returned to frontend.

CONCLUSION

OralGuard is an AI-powered Content-Based Image Retrieval system designed for oral lesion comparison. The architecture ensures fast inference, scalable deployment, and efficient similarity-based retrieval using deep learning and Docker-based hosting.